



Python



第 22 课 - 函数进阶

● 全局变量和局部变量同名问题

<pre>01. name = '蓝桥杯' 02. def demo(): 03. print(name) 04. name = '编程' 05. demo()</pre>	<pre>01. name = '蓝桥杯' 02. def demo(): 03. name = '编程' 04. print(name) 05. demo()</pre>
会报错	不会报错

✧ 报错原因

- Python 语法规则：在函数内部对不存在的变量赋值时，默认就是重新定义新的局部变量。
- 由于该局部变量和全局变量 `name` 同名，局部 `name` 就会“遮蔽”全局 `name`。
- 违反了“先定义后使用”的原则。

● 局部函数

- ✧ 如果所在函数没有返回局部函数，则局部函数的可用范围仅限于所在函数内部。

```

01. def alldef() :
02.     def indef(): ➡ 局部函数
03.         print('蓝桥杯')
04.     indef() ➡ 调用局部函数
05. alldef() ➡ 调用全局函数
>>> 蓝桥杯

```

- ✧ 如果所在函数将局部函数作为返回值，则局部函数的作用域就会扩大：既可以在所在函数内部使用，也可以在所在函数的作用域中使用。

```

01. def alldef() :
02.     def indef(): ➡ 定义局部函数
03.         print('蓝桥杯')
04.     return indef ➡ 返回局部函数
05. x = alldef()
06. x() ➡ 因为x里的值是个函数，所以要加括号
>>> 蓝桥杯

```

● 函数中的同名问题

- ✧ 如果局部函数中定义有和所在函数中变量同名的变量，也会发生“遮蔽”的问题。
- ✧ 可以使用 `nonlocal` 关键字破解。

```

01. def alldef():
02.     name = '所在函数中的name'
03.     def indef():
04.         nonlocal name
05.         print(name)
06.         name = '局部函数中的name'
07.     indef()
08. alldef()
>>> 所在函数中的name

```

● 函数高级使用方法

1. 直接将函数赋值给其它变量。

这样做的效果是，程序中也可以用其他变量来调用该函数，更加灵活。

2. 将函数以参数的形式传入其他函数中

通过使用函数作为参数，可以在调用函数时动态传入函数，从而实现动态改变函数中的部分实现代码，在不同场景中赋予函数不同的作用。

3. 函数的返回值也为函数

通过返回值为函数的形式，可以扩大局部函数的作用域。